



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)  
'walls': list(self.walls)  
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The State Space is the physical or logical configuration of the environment (e.g., the overall grid layout, wall locations, food positions, and the agent's current coordinates). It represents all possible physical states the world can be in. The Search Tree, on the other hand, is an abstract mathematical structure generated locally by the agent as it explores the State Space during its planning phase. It contains "nodes" where each node represents a specific path from the initial state to a given state. Because there can be multiple distinct paths to reach the same physical cell, the same state from the State Space can appear multiple times as different nodes within the Search Tree.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set solves the problem of redundant exploration and cycles by keeping track of which physical states (grid coordinates) the agent has already visited during its search. Without this set, a Depth-First Search (DFS) agent would easily fall into an infinite loop—for instance, endlessly moving back and forth between two adjacent empty cells (Left, Right, Left, Right). This would either exhaust the computer's stack memory or cause the simulation to run forever without ever finding the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

Breadth-First Search (BFS) is guaranteed to be optimal in this specific grid because it explores nodes level-by-level in terms of depth. Since every movement (Up, Down, Left, Right) has an identical cost of 1, exploring by depth naturally means exploring by path cost. The first time BFS discovers a food node, it is guaranteed to have taken the absolute fewest steps possible to get there. DFS, in contrast, dives as deep as possible down a single path until it hits a dead end or arbitrarily finds the food. It does not look for the shortest path; it simply returns the first path it stumbles upon, which usually results in highly erratic, winding, and suboptimal routes.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

According to the complexity formulas, BFS has a space complexity of $O(b^d)$, where b is the branching factor (up to 4 in this grid) and d is the depth of the shallowest solution. This means its memory usage grows exponentially because BFS must simultaneously keep the entire frontier of the current depth in memory (the queue). In a massive 1000x1000 grid, this would quickly consume all available RAM and crash. DFS, however, has a much lower space complexity of $O(b \times m)$, where m is the maximum depth. It only needs to store a single current path from the root to the active leaf node (along with the unexpanded siblings of nodes on that path), making DFS extremely memory-efficient compared to BFS.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING